

Cahier des charges — ARGUS-SOC

Mini-SOC intelligent augmenté par l'IA pour la surveillance d'un endpoint Windows

Version	2.0 (spec d'implémentation)
Auteur	Yassine Essid
Type	PFE — Plateforme DevSecOps / Blue Team
Destinataire de ce document	Claude Code (agent d'implémentation) + jury PFE
Codename	ARGUS (le géant aux cent yeux — surveillance ; nom de travail, à valider)
Base v1	Cahier des charges v1.0 « Mini SOC Intelligent avec IA »

Note de cadrage (à lire en premier). La v1 listait l'intégralité des fonctionnalités d'un SOC commercial (9 modules, ~15 familles de détection, 5 outils de Threat Intel, Suricata + Zeek + Wazuh + Prometheus + Grafana + Traefik...). C'est un excellent inventaire de la *cible finale*, mais c'est **infaisable en un PFE**. Cette v2 fait deux choses : (1) elle **recadre le périmètre** en MVP réaliste + roadmap ; (2) elle **réoriente la valeur du projet** — non pas reconstruire un SIEM (Wazuh le fait déjà, mieux), mais **construire la couche d'intelligence artificielle et l'expérience analyste par-dessus**. C'est là qu'est la contribution originale, soutenable devant un jury, et alignée sur ce que fait réellement le marché en 2026.

0. Comment Claude Code doit utiliser ce document

Ce cahier des charges est la **source de vérité** du projet. Il est écrit pour être déposé à la racine du dépôt et consommé par un agent de code.

Conventions imposées :

- Développement **phase par phase** (§11). Une phase n'est pas commencée tant que le *test gate* de la précédente n'est pas vert.
- Chaque phase se termine par une **Definition of Done (DoD)** vérifiable par une commande ou un test automatisé — pas par « ça a l'air de marcher ».
- Tout code livré est accompagné de tests. Objectif de couverture backend $\geq 70\%$ sur la logique métier (hors intégrations externes).
- Aucun secret en dur.** Tout passe par `.env` / variables d'environnement, jamais commité.
- Le projet doit **démarrer d'une seule commande** : `docker compose up`. Si ce n'est pas le cas à la fin d'une phase, la phase n'est pas terminée.

- Chaque module IA doit être **traçable** : toute affirmation produite par un LLM référence la ou les preuves (alerte, log, IOC) qui la fondent. Pas de « boîte noire ».
- Fichier `CLAUDE.md` à créer en phase 0 : il résume la stack, les commandes clés, l'arborescence et les règles ci-dessus, pour que chaque session d'implémentation reparte du bon contexte.

Ordre de lecture recommandé pour l'agent : §2 (décision d'architecture) → §3 (périmètre) → §4-7 (architecture, stack, données, API) → §8 (couche IA, le cœur) → §11 (plan) → §12 (prompts prêts à l'emploi).

1. Vision, positionnement et inspiration marché

1.1 Le problème réel

Un SOC ne souffre pas d'un manque d'alertes — il en croule sous. Le goulot d'étranglement, c'est le **triage** : distinguer, parmi des centaines d'événements bruts, les quelques-uns qui constituent une vraie attaque, comprendre l'enchaînement, et décider quoi faire. C'est un travail répétitif, chronophage et qui épuise les analystes (*alert fatigue*).

1.2 Le positionnement d'ARGUS

ARGUS n'est **pas un énième SIEM**. C'est une **couche analyste augmentée par l'IA** qui se pose *au-dessus* d'un SIEM open source (Wazuh) et qui transforme :

- une **file d'alertes brutes** → en une **file de verdicts** (l'analyste valide des enquêtes déjà faites, au lieu de tout investiguer à la main) ;
- des **dizaines d'alertes isolées** → en quelques **récits d'attaque** corrélés et racontés en langage clair ;
- un incident → en une **explication + causes + remédiation** exploitables, chaque affirmation étant sourcée.

1.3 Ce que fait le marché (2026) — inspiration à assumer

Synthèse de ce qui différencie aujourd'hui les plateformes de référence, et ce qu'on en retient pour ARGUS :

Acteur	Idée-force à reprendre
Elastic Security — Attack Discovery	Le LLM collapse des centaines d'alertes en quelques « attaques qui comptent » , sous forme de <i>récit d'attaque</i> (attack chain) avec une interface qui explique. L'AI Assistant produit des réponses citées (on sait quelle donnée a servi). Support LLM local pour environnements souverains/air-gapped. → C'est le pattern central d'ARGUS.
Microsoft Security Copilot / SOC agentique	Basculer vers un SOC agentique : des agents spécialisés couvrent triage, investigation, réponse ; l'humain garde le contrôle. Le triage agentique du phishing détecte plusieurs fois plus d'alertes malveillantes qu'un analyste seul. → Justifie une architecture multi-agents.
Prophet Security	Plusieurs agents spécialisés en parallèle (triage, enrichissement, test d'hypothèse, synthèse) qui « confrontent » leurs conclusions avant de produire un rapport auditable où chaque affirmation est cliquable jusqu'à la preuve. Posture par défaut conservatrice (recommande, l'humain agit). → Modèle de notre orchestration + niveaux d'autonomie.
Dropzone AI	Analyste SOC autonome de niveau Tier-1 : enquête chaque alerte, produit un rapport « décision-ready » en anglais clair, avec une Context Memory qui apprend l'environnement. Métrique-clé : réduction ~85 % du travail manuel d'investigation. → Notre RAG « mémoire de contexte » + KPIs MTTC.

Fil rouge du marché : *agents spécialisés + récit d'attaque corrélé + sortie sourcée/auditable + niveaux d'autonomie avec humain dans la boucle + mémoire de contexte.* ARGUS reprend exactement ces cinq principes, en version open source et à l'échelle d'un endpoint.

2. Décision d'architecture clé (regard senior)

■ **On ne reconstruit pas le SIEM. On construit l'intelligence par-dessus.**

Fait technique déterminant : Wazuh (version stable **4.14.5**, avril 2026) est un SIEM/XDR open source complet qui embarque *déjà*, prêt à l'emploi :

- l'indexeur **OpenSearch** (stockage + recherche des événements) ;
- le **mapping MITRE ATT&CK natif** (chaque règle porte ses identifiants de technique, les alertes arrivent déjà étiquetées **T1055**, **T1003**...) ;
- l'intégration **VirusTotal**, la détection de vulnérabilités, le **File Integrity Monitoring**, la **réponse active** (isolation, blocage) ;
- l'enrôlement d'agents Windows/Linux, chiffré (AES-256).

Conséquence pour le PFE : réécrire un collecteur d'événements, un moteur de corrélation, un moteur de règles Sigma, un mapping MITRE... serait redondant, fragile, et n'apporterait **aucune originalité**. La valeur, la difficulté et la nouveauté du projet sont **entièrement** dans :

1. **La couche IA multi-agents** (triage, enquête, remédiation, rapports, assistant conversationnel).
2. **Le dashboard analyste moderne** (paradigme « file de verdicts », récits d'attaque, preuves cliquables) — Wazuh a un dashboard OpenSearch fonctionnel mais générique et austère ; ARGUS propose une UX pensée pour la décision.
3. **L'orchestration et l'intégration** propres (API backend, base applicative, temps réel).

Décision : Wazuh = socle de collecte/détection (composant tiers déployé). ARGUS = couche IA + backend + dashboard (le code du PFE). Cette décision réduit drastiquement le risque projet et clarifie la contribution soutenable.

3. Périmètre : MVP réaliste + roadmap (MoSCoW)

Priorisation explicite. Claude Code implémente d'abord **tout le MUST**, puis attaque le SHOULD si le temps le permet.

MUST — le MVP (le PFE minimum viable et défendable)

- ✓ Déploiement Wazuh single-node + **1 endpoint Windows** (Sysmon + agent Wazuh) en lab.
- ✓ Backend **FastAPI** qui interroge OpenSearch (alertes, événements) via l'API Wazuh.
- ✓ **Base applicative PostgreSQL** (utilisateurs, incidents, verdicts, rapports, cache d'enrichissement).
- ✓ **Auth JWT + RBAC** (rôles : `admin`, `analyst`, `viewer`).
- ✓ Dashboard **Next.js** : vue temps réel des alertes, détail d'alerte, **file de verdicts**, timeline d'incident.
- ✓ **Couche IA — 4 capacités cœur** :
 1. **Triage & récit d'attaque** : corrélér les alertes liées en un incident raconté (attack narrative) avec score de risque.
 2. **Explication d'alerte** : description, cause, technique MITRE, impact, remédiation, commandes utiles — **sourcée**.
 3. **Assistant SOC conversationnel** (chat) avec RAG sur les données de l'incident + MITRE.
 4. **Génération de rapport d'incident PDF**.
- ✓ **Threat Intel — 2 sources faciles** : VirusTotal (déjà côté Wazuh) + AbuseIPDB (réputation IP), avec cache en base.
- ✓ **Choix du LLM** : abstraction provider → OpenAI **ou** Anthropic **ou** Ollama local (mode air-gapped).
- ✓ **Notifications temps réel** : alerte push sur incident critique vers Slack / Teams / Discord / email (webhooks configurables). (*quick win*)
- ✓ **Carte de couverture MITRE** : visualisation des techniques détectées vs non couvertes (donnée déjà produite par Wazuh, à afficher). (*quick win*)
- ✓ **Déobfuscation IA de commandes** : coller une commande encodée (PowerShell base64, etc.) → l'IA la décode et l'explique, de façon sûre et sans exécution. (*quick win*)

- ✓ **Dashboard KPI exécutif** : MTTD, MTTR/MTTC, taux de réduction du bruit, précision du verdict IA. (*quick win*)
- ✓ **Docker Compose** : tout démarre en une commande. **README + manuel utilisateur**.
- ✓ **Jeu d'essai défensif** : validation des détections via **Atomic Red Team** (§14).

SHOULD — extensions à valeur (après la phase 6 si le temps le permet)

- **Réponse active avec approbation (niveau L2)** : isolation de l'hôte compromis via Wazuh active-response, **déclenchée après validation humaine**, action journalisée et réversible. On passe de « détecter » à « agir » (§8.4). (*ajout à valeur*)
- **Boucle de feedback verdict → mémoire** : la décision humaine (confirmation/rejet d'un verdict) alimente le RAG ; le système apprend l'environnement au fil du temps (« context memory ») (§8.3). (*ajout à valeur*)
- **Génération de règle Sigma par IA** : lorsqu'un trou de détection est identifié, l'IA propose une règle Sigma prête à relire, bouclant le cycle détecter → corriger (§8.7). (*ajout à valeur*)
- Niveaux d'autonomie L0→L2 avec *gates* d'approbation (§8.4).
- MFA (TOTP).
- Rapports journalier / hebdomadaire automatiques.

COULD — bonus démonstratif

- Cartographie géo des IP, top attaques, chronologie enrichie.
- Détection comportementale légère (scoring d'anomalie sur fréquence/volume).

WON'T (hors périmètre PFE — roadmap future, à citer dans le mémoire)

- Multi-endpoints à grande échelle, agents Linux/macOS, ML de détection entraîné maison, SOAR complet, forensic disque/mémoire (Volatility), Kubernetes/cloud multi-provider, MISP/OpenCTI/Suricata/Zeek/Grafana. **Antivirus, EDR kernel, firewall** restent explicitement hors périmètre (comme en v1).

Règle d'or de scope : si une fonctionnalité existe déjà dans Wazuh, on l'*utilise*, on ne la réimplémente pas. Le code ARGUS ne produit que ce que Wazuh ne fait pas : l'intelligence et l'UX de décision.

4. Architecture cible

4.1 Vue d'ensemble (flux de données)



4.2 Composants et responsabilités

Composant	Rôle	Qui l'écrit
Wazuh Manager + OpenSearch	Collecte, détection, indexation, MITRE, TI de base	Tiers (déployé, pas codé)
Wazuh Agent + Sysmon	Télémétrie sur l'endpoint Windows	Tiers (configuré)
Backend FastAPI	API, auth, orchestration IA, enrichissement, temps réel	ARGUS (PFE)
PostgreSQL	Utilisateurs, incidents, verdicts, rapports, cache	ARGUS (PFE)
Redis	Cache TI, file de tâches asynchrones (Celery/ARQ)	ARGUS (PFE)
LLM provider (abstrait)	Raisonnement IA	Externe / local
Frontend Next.js	Dashboard analyste	ARGUS (PFE)

5. Stack technique figée (versions épinglées)

Épingler les versions évite les dérives de dépendances en cours de projet. Claude Code utilise **exactement** ces versions sauf blocage justifié.

Socle SIEM (tiers)

- Wazuh **4.14.5** (single-node : manager + indexer + dashboard)
- Sysmon (config de base type SwiftOnSecurity), Wazuh Agent 4.14.x sur Windows 10/11 (VM)

Backend

- Python **3.12**, FastAPI **0.115+**, Uvicorn
- SQLAlchemy 2.x + Alembic (migrations), Pydantic v2
- **LangGraph** (orchestration multi-agents) + LangChain-core
- Celery **ou** ARQ + Redis (tâches asynchrones : enquêtes, enrichissement, rapports)
- `python-jose` (JWT), `passlib[bcrypt]`
- `reportlab` ou `weasyprint` (PDF)
- Clients : `opensearch-py`, httpx

Frontend

- Next.js **15** (App Router), React 19, TypeScript
- Tailwind CSS + composants shadcn/ui

- TanStack Query (data fetching), Recharts (graphes), React Flow (récit d'attaque en graphe), socket.io-client
- Auth : cookies httpOnly + refresh token

Data

- PostgreSQL 16, Redis 7

IA / LLM

- Abstraction provider maison (`LLMProvider` interface) : implémentations OpenAI, Anthropic, **Ollama** (modèle local, ex. `llama3.1` ou `mistral`)
- Embeddings + **pgvector** (extension PostgreSQL) pour le RAG — évite d'ajouter une base vectorielle séparée

DevOps

- Docker + Docker Compose, GitHub + GitHub Actions (lint + tests)
 - Ruff + Black (Python), ESLint + Prettier (TS)
-

6. Modèle de données (PostgreSQL applicatif)

⚠️ OpenSearch (propriété de Wazuh) stocke les alertes/événements bruts. PostgreSQL ne stocke **que l'état applicatif d'ARGUS** — on ne recopie pas les logs bruts, on référence les alertes par leur `wazuh_alert_id`.

Tables MVP :

Table	Champs clés	Rôle
users	id, email, hashed_password, role, mfa_secret?, created_at	Comptes
roles	(enum applicatif : admin/analyst/viewer)	RBAC
incidents	id, title, status, risk_score, mitre_tactics[], narrative_md, created_at, assigned_to	Un « récit d'attaque » = un incident
incident_alerts	incident_id, wazuh_alert_id, rule_id, mitre_ids[], raw_ref	Lien incident ↔ alertes Wazuh corrélées
verdicts	id, incident_id, ai_disposition (malicious/benign/suspicious), ai_confidence, ai_reasoning_md, human_decision?, decided_by?, decided_at?	File de verdicts (IA propose, humain tranche)
ai_evidence	id, incident_id, claim, source_type (alert/log/ioc/mitre), source_ref, url?	Traçabilité : chaque affirmation IA ↔ sa preuve
reports	id, incident_id?, type (incident/daily/weekly), period, pdf_path, generated_at	Rapports générés
iocs	id, type (ip/hash/domain), value, ti_verdict, ti_source, cached_at	Cache Threat Intel
audit_log	id, user_id, action, target, ip, at	Journal d'audit (sécurité)
kb_documents	id, content, embedding (vector), source, metadata	RAG (MITRE, contexte org, incidents passés)
notification_channels	id, type (slack/teams/discord/email), webhook_url/config, min_severity, enabled	Config des canaux de notification
notifications	id, incident_id, channel_id, status (sent/failed), sent_at	Historique des envois
response_actions	id, incident_id, action_type (isolate_host/...), status (pending/approved/executed/reverted),	Réponse active L2 sous

Table	Champs clés	Rôle
	requested_by, approved_by, executed_at	approbation (§8.4)
<code>sigma_rules</code>	id, incident_id?, yaml, status (draft/reviewed), created_by (ai/human)	Règles Sigma générées par IA (§8.7)

Migrations gérées par Alembic. Table `kb_documents` utilise `pgvector`. La boucle de feedback (§8.3) ne crée pas de nouvelle table : elle réutilise `verdicts.human_decision` comme source d'apprentissage vers `kb_documents`.

7. Contrats d'API (extrait — le backend expose une API stable au frontend)

Convention : REST sous `/api/v1`, réponses JSON, erreurs RFC 7807. WebSocket sous `/ws`.

```
# Auth
POST    /api/v1/auth/login          → { access_token, refresh_token }
POST    /api/v1/auth/refresh
GET     /api/v1/auth/me

# Alertes (proxy lecture vers OpenSearch, normalisé)
GET     /api/v1/alerts?severity=&from=&to=&mitre=&page= → liste paginée
normalisée
GET     /api/v1/alerts/{id}        → détail +
événements liés

# Incidents / récits d'attaque
GET     /api/v1/incidents?status=&risk_min=
GET     /api/v1/incidents/{id}     → { narrative_md, alerts[], graph,
mitre[], evidence[] }
POST    /api/v1/incidents/{id}/reanalyze → relance l'orchestration IA (async →
job_id)

# Verdicts (file de décision)
GET     /api/v1/verdicts?status=pending
POST    /api/v1/verdicts/{id}/decide → { human_decision, note }

# IA
POST    /api/v1/ai/explain/{alert_id} → explication sourcée (async)
POST    /api/v1/ai/chat                → { incident_id?, message } → réponse
+ citations[]
POST    /api/v1/ai/report/{incident_id} → génère PDF (async → job_id)
POST    /api/v1/ai/deobfuscate         → { payload } → { decoded,
explanation, mitre } (quick win)
POST    /api/v1/ai/sigma/{incident_id} → génère une règle Sigma proposée
(SHOULD)

# Threat Intel
GET     /api/v1/ti/ip/{ip}          → réputation (cache-first)
GET     /api/v1/ti/hash/{sha256}

# Couverture MITRE & KPIs (quick wins)
GET     /api/v1/mitre/coverage       → techniques détectées vs non
couvertes
GET     /api/v1/metrics/kpis?from=&to= → { mtttd, mtrr, noise_reduction,
verdict_accuracy }

# Notifications (quick win)
GET     /api/v1/notifications/channels
POST    /api/v1/notifications/channels → { type, webhook_url, min_severity }
POST    /api/v1/notifications/test/{id} → envoi de test
```

```
# Réponse active L2 (SHOULD – sous approbation)
POST    /api/v1/response/{incident_id}/isolate → crée une action pending
POST    /api/v1/response/actions/{id}/approve → validation humaine →
exécution
POST    /api/v1/response/actions/{id}/revert → annulation

# Jobs asynchrones
GET     /api/v1/jobs/{job_id} → { status, result? }

# Temps réel
WS      /ws/alerts → push des nouvelles alertes /
changements de verdict
```

Toute réponse IA inclut un tableau `citations[]`: `[[{ claim, source_type, source_ref }]]`.
Pas de sortie IA sans citations.

8. La couche IA — le cœur intelligent (le vrai sujet du PFE)

C'est ici que se concentrent la difficulté et l'originalité. Orchestration avec **LangGraph** (graphe d'agents), chaque agent = un nœud avec un rôle borné.

8.1 Les agents (multi-agents spécialisés)

Agent	Entrée	Sortie	Rôle
Triage Agent	lot d'alertes brutes (fenêtre glissante)	groupes corrélés + score de risque	Regroupe les alertes liées (même hôte, même processus, même chaîne temporelle, MITRE contigus) en incidents candidats . Écarte le bruit / faux positifs évidents.
Enrichment Agent	un incident candidat	incident enrichi	Résout les IOC (IP→AbuseIPDB, hash→VirusTotal), ajoute contexte hôte/utilisateur, réputation.
Investigation Agent	incident enrichi	récit d'attaque (narrative) + hypothèse + confiance	Reconstruit la <i>kill chain</i> : que s'est-il passé, dans quel ordre, quelle technique MITRE à chaque étape. Teste une hypothèse (attaque réelle vs bénin).
Remediation Agent	récit d'attaque	actions recommandées + commandes	Propose la remédiation priorisée + commandes utiles + références.
Report Agent	incident finalisé	Markdown → PDF	Rédige le rapport d'incident structuré.
SOC Assistant (chat)	question analyste + contexte incident	réponse citée	Répond en langage naturel via RAG (données incident + MITRE + KB).
Deobfuscation Agent (quick win)	payload obfusqué (PowerShell base64, chaîne encodée)	commande décodée + explication + technique MITRE	Décode et explique sans exécuter une commande suspecte. Purement analytique : aucun payload n'est lancé, on décrit ce qu'il ferait.

Pattern de référence marché : des agents spécialisés qui produisent un **récit auditable** plutôt qu'un score opaque. Chaque nœud LangGraph journalise son raisonnement.

8.2 Contrainte non négociable : traçabilité (sortie sourcée)

Chaque affirmation d'un agent doit être rattachée à une preuve (alerte, log, IOC, page MITRE) et enregistrée dans `ai_evidence`. Le frontend rend ces preuves **cliquables**. Une sortie IA sans preuve est un bug, pas une fonctionnalité.

8.3 RAG « mémoire de contexte » + boucle de feedback

Base (`kb_documents`) (pgvector) alimentée par : (a) le référentiel MITRE ATT&CK (tactiques/techniques/mitigations), (b) le contexte de l'organisation (hôtes critiques, comptes sensibles — saisi manuellement au début), (c) **les incidents passés et leurs verdicts humains**.

Boucle de feedback (ajout à valeur). Chaque fois qu'un analyste confirme ou **rejette** un verdict IA dans la file, cette décision (l'incident, la disposition proposée, la correction humaine) est indexée dans (`kb_documents`). Aux enquêtes suivantes, le Triage et l'Investigation Agent récupèrent par similarité les décisions passées sur des incidents proches → le système **apprend l'environnement au fil du temps** et réduit la récurrence des mêmes faux positifs (inspiration « Context Memory » du marché). C'est un cycle vertueux mesurable : la précision du verdict doit s'améliorer avec le nombre de décisions humaines accumulées (KPI §8.6).

8.4 Niveaux d'autonomie (humain dans la boucle) — SHOULD

Le degré d'autonomie est **configurable**, jamais imposé :

- **L0 — Observe** : l'IA explique, ne décide rien.
- **L1 — Recommande** (défaut) : l'IA produit un verdict + une remédiation, **l'humain tranche** dans la file de verdicts.
- **L2 — Agit sous approbation** : l'IA peut déclencher une action Wazuh (isolation d'hôte) **après clic de validation** d'un analyste. Action **journalisée et réversible**.
- L3 (autonome total) : **hors périmètre PFE** — mentionné comme perspective.

Pour chaque action : indiquer clairement *qui l'a décidée, est-elle réversible, est-elle journalisée*. C'est la question de confiance centrale du marché.

8.5 Abstraction LLM (souveraineté / mode local)

Interface (`LLMProvider`) (méthodes (`complete`), (`chat`), (`embed`)). Implémentations OpenAI / Anthropic / **Ollama local**. Le mode Ollama permet une démo **air-gapped** — argument fort pour le jury (données sensibles jamais envoyées à un tiers). Le provider est choisi par variable d'environnement.

8.6 KPIs à instrumenter (pour le mémoire et la démo)

- **MTTD** (temps de détection), **MTTC/MTTR** (temps jusqu'à conclusion/remédiation)
- % d'alertes auto-triées, taux de réduction du bruit (alertes → incidents)
- Taux de faux positifs écartés, précision du verdict IA vs décision humaine Ces chiffres, mesurés sur le jeu d'essai Atomic Red Team, donnent une évaluation quantifiée du système — très apprécié d'un jury.

8.7 Génération de règle Sigma par IA (ajout à valeur — SHOULD)

Quand un incident révèle un comportement pour lequel la couverture MITRE (§9.2) montre un trou de détection, un agent propose une **règle Sigma** (YAML standard, portable vers d'autres SIEM) décrivant la détection manquante. La règle est **toujours en statut**

draft : un humain la relit et la valide avant tout usage. Cela boucle le cycle *détecter* → *identifier le manque* → *corriger*, et démontre une plateforme qui ne fait pas que constater mais améliore sa propre détection.

8.8 Notifications proactives (quick win)

Sur incident dépassant un seuil de sévérité configurable, le backend pousse une notification vers les canaux activés (Slack / Teams / Discord via webhook, email via SMTP). Le message contient le titre du récit, le score de risque, les techniques MITRE et un lien direct vers l'incident dans ARGUS. Traitement **asynchrone** (worker) pour ne pas bloquer l'ingestion ; échecs journalisés et rejoués. Transforme ARGUS d'un outil qu'on consulte en un outil qui **alerte**.

9. UX / UI — pensé pour la décision

9.1 Principe directeur

On ne conçoit pas un « tableau de bord de logs » de plus. On conçoit une **surface de décision** : l'analyste doit, en un écran, comprendre *quelle est la menace, pourquoi, et quoi faire*.

9.2 Écrans clés (MVP)

1. **File de verdicts (écran d'accueil)** — le paradigme central. Liste d'incidents que l'IA a déjà investigués, triés par risque. Chaque ligne : titre du récit, score, techniques MITRE, disposition IA proposée, boutons *Confirmer* / *Rejeter* / *Enquêter*. L'analyste valide, il ne repart pas de zéro.
2. **Détail d'incident / récit d'attaque** — le récit en langage clair, la **kill chain en graphe** (React Flow) étape par étape, chaque étape reliée à ses alertes et à sa technique MITRE. Panneau latéral « Preuves » cliquables. Bloc remédiation.
3. **Flux d'alertes temps réel** — WebSocket, filtrable (sévérité, MITRE, hôte). Pour l'analyste qui veut la vue brute.
4. **Assistant SOC (chat)** — panneau conversationnel contextualisé à l'incident ouvert, réponses **citées**. Inclut l'outil **déobfuscation** (coller une commande encodée → décodage + explication).
5. **Carte de couverture MITRE (quick win)** — matrice tactiques × techniques colorée : ce qui est détecté (vert), effleuré, ou non couvert (gris). Vue immédiate des forces et des trous de détection ; sert d'amorce à la génération Sigma (§8.7).
6. **Dashboard KPI exécutif (quick win)** — cartes et courbes : MTTD, MTTR/MTTC, réduction du bruit (alertes→incidents), précision du verdict IA dans le temps. C'est l'écran qui **prouve chiffres à l'appui** que le système fonctionne.
7. **Rapports** — génération et téléchargement PDF.
8. **Admin** — utilisateurs, rôles, choix du provider LLM, niveau d'autonomie, **canaux de notification** (Slack/Teams/Discord/email), config TI.

9.3 Direction visuelle

Thème sombre par défaut (standard SOC, confort en salle), accent sur la hiérarchie de sévérité (couleur = risque, jamais décorative), typographie lisible en densité d'information,

états de chargement explicites pour les tâches IA asynchrones (« l'agent enquête... »), et surtout : **la preuve est toujours à un clic**. Pas de jargon non expliqué. Un `read_me` de design tokens sera défini en phase 4 avant d'écrire les composants.

Consulter un ou deux **analystes SOC seniors** (ou enseignants spécialisés à ESPRIT) sur deux points précis : (1) l'ordre des informations dans le détail d'incident, (2) quelles actions ils veulent pouvoir faire sans quitter l'écran. Un retour terrain de 30 minutes vaut mieux que des suppositions.

10. Sécurité de la plateforme elle-même

Une plateforme de sécurité doit être exemplaire. **MUST** : JWT (courte durée) + refresh en cookie httpOnly, RBAC strict, bcrypt, **journal d'audit** de toute action sensible, validation systématique des entrées (Pydantic), rate limiting sur l'auth et les endpoints IA (coûteux), protection CSRF/XSS/SQLi (ORM paramétré), secrets hors du code, HTTPS en déploiement. **SHOULD** : MFA TOTP, chiffrement au repos des champs sensibles. Le mode LLM local (Ollama) est aussi un argument de **confidentialité des données**.

11. Plan de réalisation par phases (avec test gates)

Chaque phase a une **DoD vérifiable**. Claude Code ne passe à la phase N+1 que si le gate de N est vert.

Phase 0 — Fondations

Repo, `CLAUDE.md`, `docker-compose.yml` (postgres + redis + backend + frontend vides), CI (lint + test), `.env.example`, structure §13. **Gate** : `docker compose up` démarre tous les services ; `GET /health` répond 200 ; CI verte.

Phase 1 — Socle SIEM (lab)

Déploiement Wazuh 4.14.5 single-node ; VM Windows avec Sysmon + agent enrôlé ; vérifier que des alertes remontent dans OpenSearch. **Gate** : une action de test (ex. exécution PowerShell suspecte) génère une alerte visible dans OpenSearch, étiquetée MITRE.

Phase 2 — Backend : auth + ingestion

Modèles SQLAlchemy + migrations Alembic ; auth JWT/RBAC ; client OpenSearch ; endpoint `/api/v1/alerts` normalisé + pagination. **Gate** : tests d'intégration : login retourne un token ; un `viewer` est refusé sur un endpoint admin (403) ; `/alerts` renvoie des alertes réelles issues de Wazuh.

Phase 3 — Enrichissement Threat Intel + couverture MITRE

Clients VirusTotal + AbuseIPDB, cache Redis/Postgres (cache-first), endpoints `/ti/*`. **Endpoint** `/mitre/coverage` (quick win) : agréger les techniques vues dans Wazuh vs le référentiel. **Gate** : une IP publique connue malveillante renvoie un verdict TI ; le 2^e appel est servi depuis le cache (vérifié par mock/log) ; `/mitre/coverage` renvoie des techniques détectées et non couvertes.

Phase 4 — Frontend : shell + flux d'alertes + carte MITRE

Next.js, auth (login, refresh), layout, thème, écran flux d'alertes temps réel (WebSocket), détail d'alerte, **carte de couverture MITRE** (quick win), design tokens. **Gate** : un utilisateur se connecte, voit les alertes en direct, ouvre un détail ; une nouvelle alerte Wazuh apparaît sans rafraîchir la page ; la matrice MITRE s'affiche colorée.

Phase 5 — Couche IA : triage + explication + déobfuscation (cœur)

Abstraction (`LLMProvider`) (OpenAI/Anthropic/Ollama) ; graphe LangGraph
Triage→Enrichment→Investigation ; création d'incidents/récits ; `ai_evidence` peuplé ;
endpoint `/ai/explain` ; endpoint `/ai/deobfuscate` (quick win). **Gate** : sur un jeu d'alertes corrélées, le système produit un incident avec récit + score + techniques MITRE, et **chaque affirmation a au moins une preuve** dans `ai_evidence` ; une commande PowerShell base64 fournie est correctement décodée et expliquée (sans exécution). Sortie reproductible avec le provider Ollama (mode local).

Phase 6 — File de verdicts + chat + rapports + notifications

Écran file de verdicts (confirmer/rejeter, écrit dans `verdicts`) ; graphe kill chain (React Flow) ; chat SOC RAG (pgvector, réponses citées) ; génération PDF ; **notifications** (quick win) Slack/Teams/Discord/email sur incident critique (worker async). **Gate** : un analyste confirme un verdict (persisté + audité) ; le chat répond à « pourquoi cette alerte ? » avec citations cliquables ; un PDF est téléchargeable ; un incident critique déclenche une notification reçue sur un canal de test.

Phase 7 — Validation défensive (purple team) + dashboard KPI

Rejouer des techniques via **Atomic Red Team** (§14) ; instrumenter et **afficher le dashboard KPI exécutif** (quick win) : MTDD, MTTR, réduction du bruit, précision du verdict. **Gate** : ≥ 5 techniques MITRE distinctes rejouées, détectées, triées en incidents et correctement expliquées ; le dashboard KPI affiche des valeurs réelles calculées sur ces essais.

Phase 8 — Extensions à valeur (SHOULD, si le temps le permet)

Dans l'ordre de priorité : (1) **boucle de feedback** verdict→`kb_documents` (§8.3) ; (2) **réponse active L2** isolation d'hôte sous approbation (§8.4) ; (3) **génération Sigma** par IA (§8.7) ; puis MFA, rapports périodiques. **Gate (par item, indépendants)** : (1) après plusieurs verdicts humains, une enquête sur un incident similaire cite une décision passée. (2) une demande d'isolation crée une action `pending`, exécutée seulement après approbation, journalisée et réversible. (3) sur un trou de couverture MITRE, une règle Sigma `draft` valide est générée.

Phase 9 — Finition & livraison

Durcissement sécurité (§10), rate limiting, README + **manuel utilisateur**, schémas d'architecture, script de démo reproductible, nettoyage. **Gate** : un tiers suit le README depuis zéro et obtient une instance fonctionnelle ; démo de bout en bout scriptée qui passe.

12. Prompts d'implémentation prêts à l'emploi (format R-T-C-R-S-O)

Modèles à donner à Claude Code, une tâche à la fois. Structure : **Rôle** • **Tâche** • **Contexte** • **Références** • **Étapes** • **Sortie attendue**.

Exemple — Phase 5, nœud Investigation Agent

[RÔLE]

Tu es un ingénieur backend Python senior spécialisé en orchestration LLM (LangGraph) et en détection/réponse sécurité.

[TÂCHE]

Implémenter le nœud "Investigation Agent" du graphe LangGraph : à partir d'un incident enrichi, produire un récit d'attaque (kill chain) structuré, une hypothèse (attaque réelle / bénin), un score de confiance, et la liste des preuves associées.

[CONTEXTE]

- Voir §6 (modèle de données : incidents, incident_alerts, ai_evidence), §7 (API), §8 (agents, traçabilité obligatoire).
- L'entrée est un objet IncidentCandidate (alertes corrélées déjà enrichies).
- Le provider LLM est abstrait derrière l'interface LLMPProvider ; ne jamais appeler un SDK vendor directement.

[RÉFÉRENCES]

- Fichiers existants : app/ai/graph.py, app/ai/providers/base.py, app/models/incident.py.
- MITRE : les alertes portent déjà leurs technique IDs (champ mitre_ids).

[ÉTAPES]

1. Définir le schéma Pydantic de sortie (AttackNarrative) : steps[] (ordre, timestamp, description, mitre_id, evidence_refs[]), hypothesis, confidence (0-1), risk_score (0-100).
2. Écrire le prompt système qui impose : (a) reconstruction chronologique, (b) une technique MITRE par étape quand disponible, (c) CHAQUE affirmation référence au moins une alerte source (evidence_refs).
3. Implémenter le nœud, parser la sortie en AttackNarrative, persister l'incident + peupler ai_evidence.
4. Gérer les erreurs LLM (retry, timeout) et le cas "aucune corrélation".

[SORTIE ATTENDUE]

- Code du nœud + schéma Pydantic + tests unitaires (avec LLM mocké).
- Aucune affirmation sans preuve : un test vérifie que chaque step a au moins un evidence_ref valide.
- Fonctionne identiquement avec le provider Ollama (test paramétré).

Décliner ce gabarit pour chaque tâche importante des phases 2 à 8.

13. Structure du dépôt

```
argus-soc/
├── CLAUDE.md                # contexte agent : stack, commandes, règles
├── README.md
├── docker-compose.yml
├── .env.example
├── docs/
│   ├── cahier-des-charges.md # ce document
│   ├── architecture.md
│   └── manuel-utilisateur.md
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── core/          # config, sécurité, deps
│   │   ├── api/v1/        # routes
│   │   ├── models/        # SQLAlchemy
│   │   ├── schemas/       # Pydantic
│   │   ├── services/      # wazuh, opensearch, ti
│   │   ├── ai/
│   │   │   ├── graph.py   # orchestration LangGraph
│   │   │   ├── agents/    # triage, enrichment, investigation, ...
│   │   │   ├── providers/ # openai, anthropic, ollama
│   │   │   └── rag/        # pgvector, embeddings
│   │   └── workers/       # tâches async
│   ├── alembic/
│   └── tests/
├── frontend/
│   ├── app/                # Next.js App Router
│   ├── components/
│   ├── lib/                # api client, ws, auth
│   └── tests/
├── lab/
│   ├── wazuh/              # docker/config Wazuh single-node
│   ├── sysmon/             # config Sysmon
│   └── atomic-red-team/     # scripts de validation
└── .github/workflows/      # CI
```

14. Jeu d'essai — validation défensive (purple team)

But : **prouver que la détection et l'IA fonctionnent**, en environnement de lab isolé. On ne développe aucun outil offensif ; on **rejoue des tests standard** mappés MITRE pour valider les détections (démarche purple team classique et légitime).

- **Atomic Red Team** : rejouer des techniques atomiques mappées MITRE (ex. exécution PowerShell encodée, création de persistance, énumération de comptes) sur la VM

Windows de lab.

- Vérifier pour chacune : alerte Wazuh générée → corrélée en incident → récit correct → technique MITRE juste → remédiation pertinente.
- Environnement **strictement isolé** (réseau lab, snapshots VM), aucune donnée réelle, aucune machine de production.

(Nmap / Hydra / Metasploit de la v1 peuvent servir en bonus pour générer du trafic réseau/brute force à détecter, mais Atomic Red Team suffit largement au MVP et est plus propre à documenter.)

15. Risques & mitigations

Risque	Impact	Mitigation
Sur-scope (reproduire la v1)	Projet jamais fini	Périmètre MoSCoW strict (§3) ; règle « si Wazuh le fait, on l'utilise ».
Ressources du lab (Wazuh + OpenSearch gourmands)	Instabilité	Single-node, VM 4 vCPU / 8 Go min ; SSD ; 1 seul endpoint.
Coût/latence LLM	Démo lente ou chère	Cache, traitement async, provider Ollama local pour dev/démo.
Hallucination IA	Verdicts faux	Traçabilité obligatoire (§8.2) + humain dans la boucle (L1 par défaut).
Sortie LLM non structurée	Parsing cassé	Schémas Pydantic + validation + retry (pattern « self-correction »).
Dérive de dépendances	Build cassé en fin de projet	Versions épinglées (§5), lockfiles commités.
Déobfuscation : exécution accidentelle d'un payload	Compromission du lab	Décodage purement statique/textuel , jamais d'exécution ; l'IA décrit, ne lance rien.
Réponse active L2 : isolation erronée	Coupure indésirable	Approbation humaine obligatoire, action journalisée et réversible , jamais automatique.
Spam de notifications	Fatigue d'alerte 2.0	Seuil de sévérité configurable + regroupement par incident (pas par alerte).

16. Ce qui rendra ce PFE remarquable

1. **Un cadrage lucide** : reconnaître que Wazuh est le SIEM et construire *par-dessus* — un jury valorise le discernement d'ingénieur autant que le code.

2. **La traçabilité comme principe** : chaque conclusion IA sourcée. C'est exactement le sens dans lequel va le marché (IA auditable, pas boîte noire) et c'est démontrable.
 3. **Le paradigme file-de-verdicts** : une UX qui change le travail de l'analyste, pas juste un dashboard de logs.
 4. **Le mode 100 % local (Ollama)** : une démo souveraine, sans donnée envoyée à un tiers — argument de confidentialité fort.
 5. **Une évaluation quantifiée** : KPIs mesurés sur Atomic Red Team (MTTD, réduction du bruit, précision du verdict), pas juste « ça marche ».
 6. **Un cycle complet détecter → agir → s'améliorer** : la réponse active sous approbation (L2), la boucle de feedback qui fait apprendre le système, et la génération Sigma qui comble ses propres trous de détection montrent une plateforme vivante, pas un simple visualiseur.
-

Annexe — Correspondance avec la v1

Tous les modules v1 sont conservés comme *cible* : les modules 1-3 (users, surveillance Windows/réseau) sont couverts par Wazuh+Sysmon ; le module 4 (détection) par les règles Wazuh + MITRE natif ; les modules 5-9 (TI, corrélation, dashboard, IA, rapports) sont la **couche ARGUS**. Ce que la v2 change : elle **priorise, réattribue** ce qui revient à Wazuh vs au code du PFE, et **concentre l'effort** sur l'intelligence — là où se gagne un PFE.